2.7. Визуализация данных (библиотека matplotlib)

2. Файлы, данные, простая обработка и визуализация

В этом уроке

- Узнаем как рисовать графики
- Какие есть графики

Почему визуализация важна?

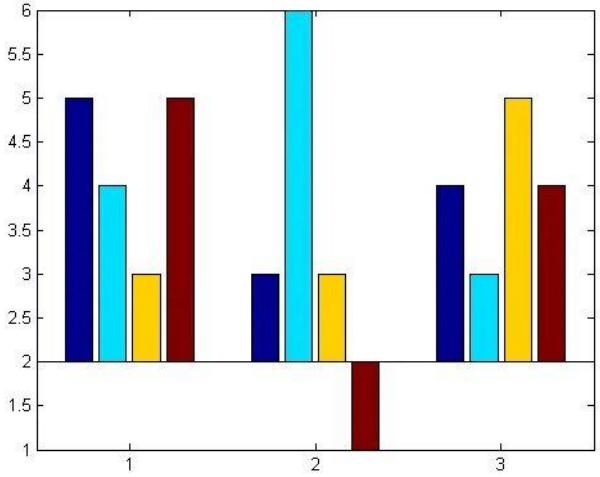
- Позволяет быстро выявлять закономерности в данных.
- Делает анализ более наглядным.
- Помогает в презентации результатов.

Визуализация данных с помощью библиотеки matplotlib

Matplotlib — это основная библиотека Python для визуализации данных.

Она позволяет строить:

- Линейные графики
- Столбчатые диаграммы (гистограммы)
- Круговые диаграммы
- Точечные графики (scatter plots)
- И многое другое



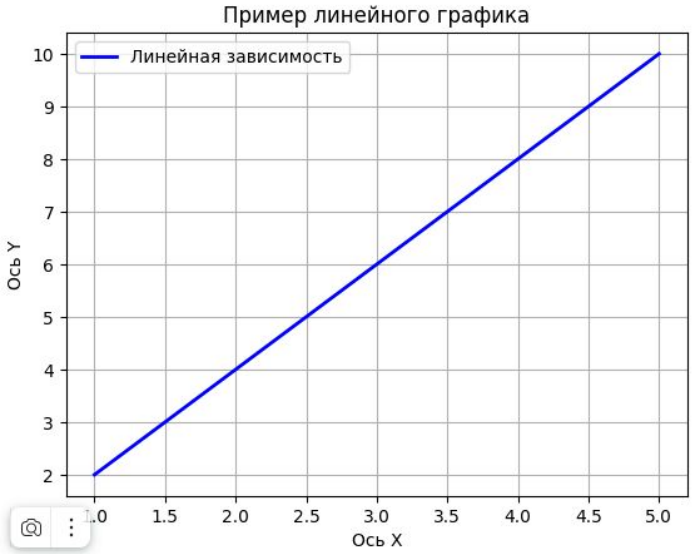
Установка

Установка:

pip install matplotlib

Пример графика:

```
import matplotlib.pyplot as plt

# Данные
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Построение графика
plt.plot(x, y, label="Линейная зависимость", color="blue", linestyle="-", linewidth=2)
plt.xlabel("Ось X")  # Подпись оси X
plt.ylabel("Ось Y")  # Подпись оси Y
plt.title("Пример линейного графика")  # Заголовок
plt.legend()  # Отображение легенды
plt.grid()  # Сетка
plt.show()  # Вывод графика
```



Линейный график (plt.plot)

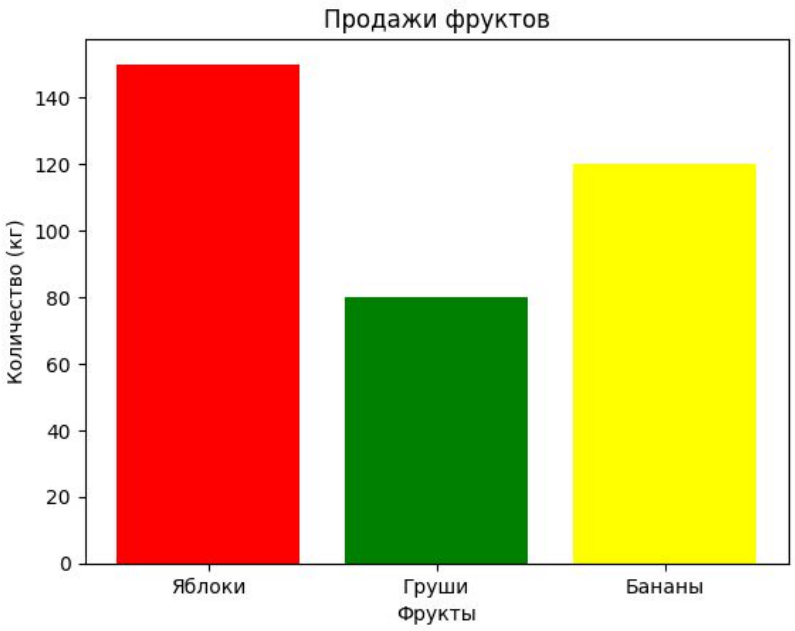
Используется для отображения изменения величины во времени.

```
days = ["Пн", "Вт", "Ср", "Чт", "Пт"]
sales = [120, 150, 90, 200, 180]

plt.plot(days, sales, marker="o", label="Продажи")
plt.title("Продажи по дням недели")
plt.xlabel("День")
plt.ylabel("Продажи (шт)")
plt.legend()
plt.show()
```

Столбчатая диаграмма (plt.bar)

Используется для отображения изменения величины во времени.

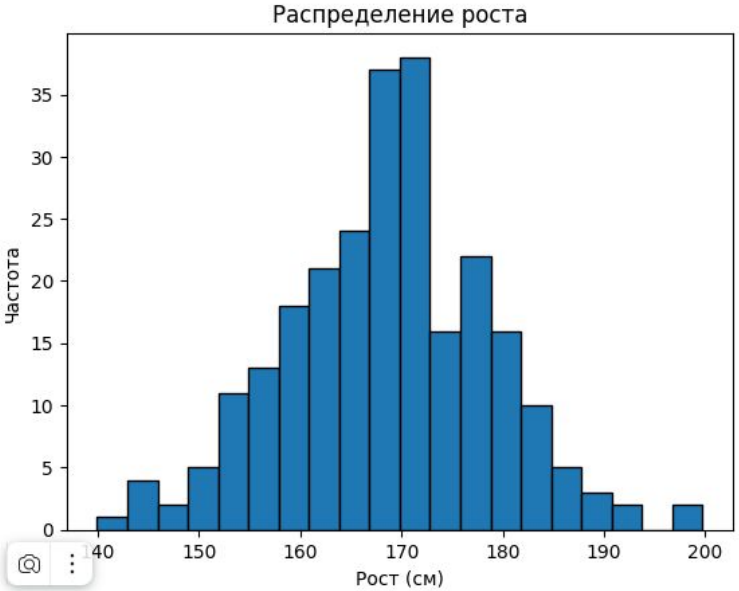
```
products = ["Яблоки", "Груши", "Бананы"]
quantities = [150, 80, 120]

plt.bar(products, quantities, color=["red", "green", "yellow"])
plt.title("Продажи фруктов")
plt.xlabel("Фрукты")
plt.ylabel("Количество (кг)")
plt.show()
```



Гистограмма (plt.hist)

Используется для отображения изменения величины во времени.

```
import numpy as np

# Генерация случайных данных
data = np.random.normal(170, 10, 250)  # Среднее = 170, std = 10, 250 точек

plt.hist(data, bins=20, edgecolor="black")
plt.title("Распределение роста")
plt.xlabel("Рост (см)")
plt.ylabel("Частота")
plt.show()
```



Круговая диаграмма (plt.pie)

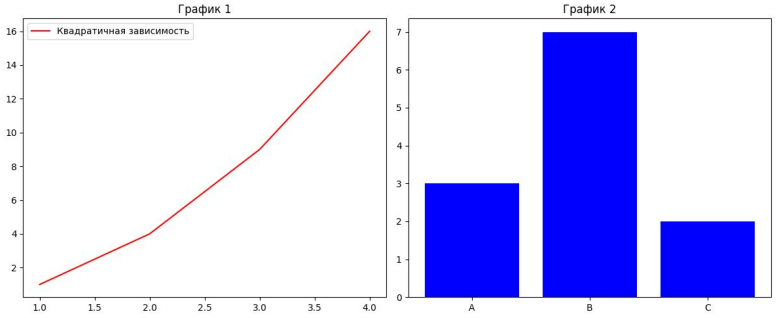
Используется для отображения изменения величины во времени.

```
import matplotlib.pyplot as plt

# Данные
categories = ["Еда", "Транспорт", "Жильё", "Развлечения", "Сбережения"]
expenses = [15000, 5000, 30000, 8000, 12000]

# Построение
plt.pie(
    expenses,
    labels=categories,
    autopct='%1.1f%%',  # Проценты с 1 знаком после запятой
    startangle=90,      # Поворот на 90 градусов
    shadow=True,        # Тень
    explode=(0.1, 0, 0, 0, 0)  # Выделяем первый сектор
)

plt.title("Распределение месячных расходов")
plt.show()
```

Размещение нескольких графиков (subplots)

Можно создавать несколько графиков в одной фигуре

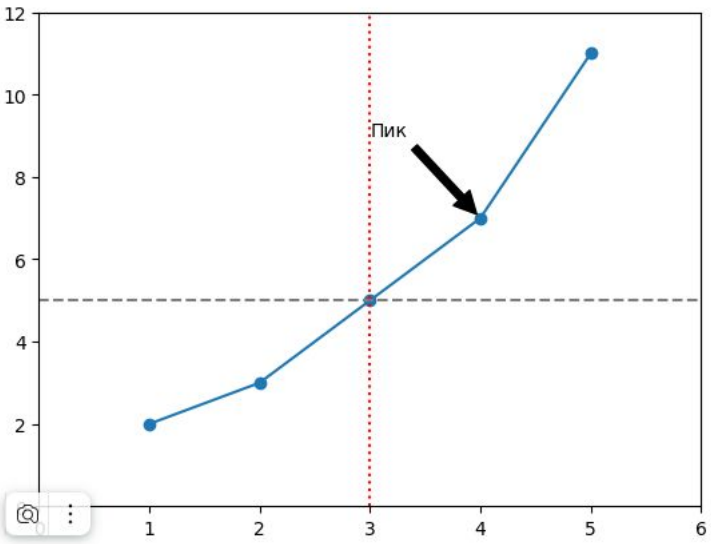
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))  # 1 строка, 2 столбца

# Первый график (линейный)
ax1.plot([1, 2, 3, 4], [1, 4, 9, 16], 'r-', label="Квадратичная зависимость")
ax1.set_title("График 1")
ax1.legend()

# Второй график (столбчатый)
ax2.bar(["A", "B", "C"], [3, 7, 2], color='blue')
ax2.set_title("График 2")

plt.tight_layout()  # Автоматическая настройка отступов
plt.show()
```



Размещение нескольких графиков (subplots)

Можно создавать несколько графиков в одной фигуре

```
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

plt.plot(x, y, marker='o')
plt.xlim(0, 6)  # Ограничение оси X
plt.ylim(0, 12)  # Ограничение оси Y

# Аннотация
plt.annotate(
    "Пик",
    xy=(4, 7),  # Координаты точки
    xytext=(3, 9),  # Координаты текста
    arrowprops=dict(facecolor='black', shrink=0.05)  # Стрелка
)

plt.axhline(y=5, color='gray', linestyle='--')  # Горизонтальная линия
plt.axvline(x=3, color='red', linestyle=':')  # Вертикальная линия
plt.show()
```

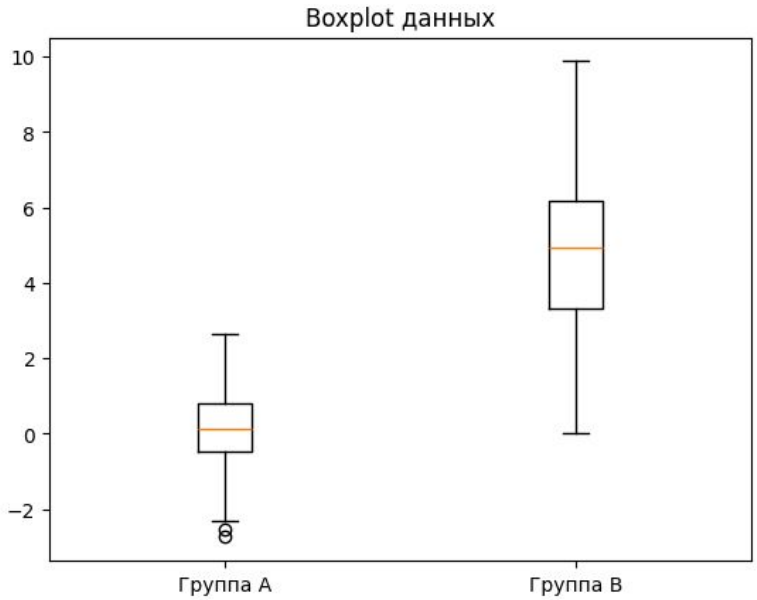


Графики для анализа распределения

Boxplot (ящик с усами)

Показывает медиану, квартили и выбросы:

```
data = [np.random.normal(0, 1, 100), np.random.normal(5, 2, 100)]
plt.boxplot(data, labels=["Группа A", "Группа B"])
plt.title("Boxplot данных")
plt.show()
```

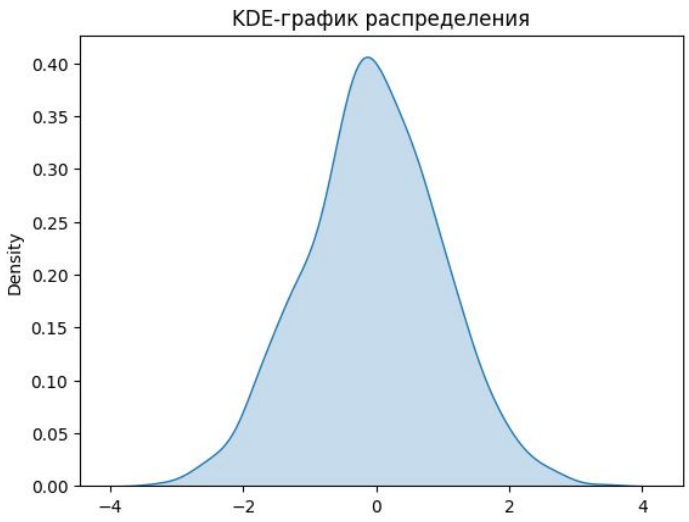


Графики для анализа распределения

График плотности (kdeplot из seaborn)

Показывает оценку плотности распределения:

```
import seaborn as sns  # Библиотека для улучшенной визуализации

data = np.random.normal(0, 1, 1000)
sns.kdeplot(data, shade=True)
plt.title("KDE-график распределения")
plt.show()
```



Итоги урока

- Узнали как рисовать графики
- Какие есть графики